Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Освітній проєкт №1

Методи і технології паралельного програмування

Тема: Моделювання броунівського руху за допомогою багатопотокового програмування

Виконав:

студент групи ІП-51мн

Панченко С. В.

Перевірив:

Лесик В. О.

Київ 2026

ЗМІСТ

1 Мета роботи 3
2 Завдання до виконання 4
3 Обґрунтування обрання фреймворку 5
4 Опис рішень та критичних моментів 6
4.1 Задача 1. Стан гонитви та взаємодія потоків 6
4.2 Задача 2. Генерація випадкових чисел 6
4.3 Задача 3. Зняття зліпків та верифікація збереження маси 6
5 Результати роботи програми 7
6 Висновки 8
А Лістинг коду 9
А.1 my/src/bin/main.rs 9
А.2 my_shaders/src/lib.rs 40

1 МЕТА РОБОТИ

Засвоїти моделі паралелізму. Реалізувати паралельну симуляцію броунівського руху. Дослідити проблеми синхронізації та розділення даних. Проаналізувати продуктивність підходу з використанням обчислювальних шейдерів.

2 ЗАВДАННЯ ДО ВИКОНАННЯ

1. Змоделювати броунівський рух 100 000 домішкових частинок у двовимірному кристалі.
2. Реалізувати масштабування за принципом “один потік для кожного атома”.
3. Вирішити проблему стану гонитви при одночасній зміні клітинки кристала кількома потоками.
4. Вирішити проблему доступу потоків до генератора випадкових чисел.
5. Реалізувати механізм зняття зліпків стану системи та перевірки збереження загальної кількості частинок без зупинки симуляції.

3 ОБҐРУНТУВАННЯ ОБРАННЯ ФРЕЙМВОРКУ

Для реалізації використано мову Rust та графічний API Vulkan. Створення 100 000 потоків на рівні операційної системи (CPU) супроводжується значними накладними витратами на перемикання контексту. Використання GPU та обчислювальних шейдерів вирішує цю проблему, оскільки архітектура відеокарти підтримує необхідну кількість одночасних потоків. Кожен атом обробляється окремим потоком шейдера, що забезпечує розпаралелювання 1 до 1.

4 ОПИС РІШЕНЬ ТА КРИТИЧНИХ МОМЕНТІВ

4.1 Задача 1. Стан гонитви та взаємодія потоків

При русі частинок виникає ситуація, коли кілька потоків одночасно оновлюють лічильник щільності в одній клітинці масиву. Без синхронізації це призводить до втрати частинок. Оскільки використання м'ютексів на GPU не реалізовано в даній архітектурі, проблему вирішено за допомогою апаратно-підтримуваної атомарної операції додавання. Це забезпечує консистентність даних при колізіях без блокування інших потоків.

4.2 Задача 2. Генерація випадкових чисел

Звернення 100 000 потоків до глобального генератора випадкових чисел створює блокування. Проблему вирішено шляхом децентралізації генератора. Використано алгоритм Xorshift32. Поточний стан зберігається індивідуально в структурі кожної частинки. Для визначення напрямку руху використовується побітовий зсув. Додано вирівнювання пам'яті для структури частинки, що запобігає помилкам читання у відеопам'яті.

4.3 Задача 3. Зняття зліпків та верифікація збереження маси

Процес рендерингу функціонує як безперервний потік візуальних зліпків стану. Для підтвердження збереження кількості частинок реалізовано механізм асинхронного зворотного читання. Із заданою періодичністю вміст сітки копіюється з відеопам'яті в оперативну пам'ять. На наступному кадрі центральний процесор зчитує ці дані, розраховує суму значень усіх клітинок і порівнює її з початковою кількістю частинок. Це дозволяє виконувати перевірку без зупинки графічного конвеєра.

5 РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

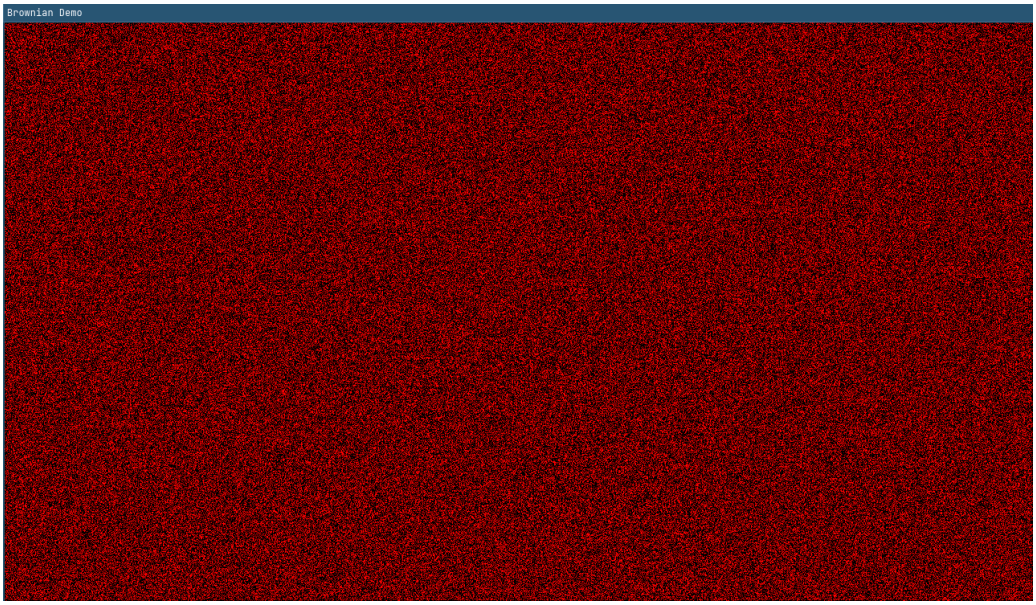

Рисунок 5.1 — Візуалізація броунівського руху в двовимірній сітці

```
Frame 2000: Total Particles = 800000
Frame 2100: Total Particles = 800000
Frame 2200: Total Particles = 800000
Frame 2300: Total Particles = 800000
Frame 2400: Total Particles = 800000
Frame 2500: Total Particles = 800000
Frame 2600: Total Particles = 800000
Frame 2700: Total Particles = 800000
Frame 2800: Total Particles = 800000
Frame 2900: Total Particles = 800000
```

Рисунок 5.2 — Вивід консолі: підтвердження збереження кількості частинок

6 ВИСНОВКИ

У ході виконання проєкту реалізовано багатопотокову симуляцію броунівського руху. Встановлено, що перенесення обчислень з CPU на GPU за допомогою Vulkan дозволяє вирішити проблему масштабування для великої кількості потоків. Використання атомарних операцій усунуло стан гонитви при модифікації масиву даних. Децентралізація генератора випадкових чисел дозволила уникнути блокувань при синхронізації. Реалізація механізму асинхронного читання забезпечила можливість контролю цілісності даних симуляції без зниження продуктивності.

Додаток А
Лістинг коду

A.1 my/src/bin/main.rs

```
#![allow(non_snake_case)]
#![allow(non_camel_case_types)]
#![allow(non_upper_case_globals)]
#![allow(unused_variables)]
#![allow(dead_code)]

use nix;
use vk;

use my_shaders::{Particle, PushConstants, Xorshift32};
use std::os::fd::{AsFd, AsRawFd, BorrowedFd, FromRawFd, IntoRawFd};

// --- SIMULATION PARAMETERS ---
const GRID_WIDTH: u32 = 1280;
const GRID_HEIGHT: u32 = 720;
const NUM_PARTICLES: u32 = 800_000;
const _: () = const {
    assert!(NUM_PARTICLES < GRID_WIDTH * GRID_HEIGHT);
};
const MAX_DENSITY: u32 = 2;

// --- CORE SKELETON MACROS & TRAITS ---

struct Defer<T: FnMut()> {
    func: T,
}
impl<T: FnMut()> Defer<T> {
    fn new(func: T) -> Self {
        Self { func }
    }
}
impl<T: FnMut()> Drop for Defer<T> {
    fn drop(&mut self) {
        (self.func)();
    }
}

macro_rules! c_str {
    ($s:literal) => {
        concat!($s, "\0").as_ptr() as *const std::ffi::c_char
    };
}
```

```rust
macro_rules! defer {
    ($($item:tt)*) => { let _defer = Defer::new(|| { $($item)* }); }
}

macro_rules! contains_bits {
    ($target:expr, $required:expr) => {
        ($target & $required) == $required
    };
}

macro_rules! vk_enumerate {
    ($func:expr) => { vk_enumerate!($func,) };
    ($func:expr, $( $arg:expr ),*) => {
        {
            let mut count = 0;
            unsafe {
                $func($($arg,)* &mut count, std::ptr::null_mut());
                let mut data = Vec::with_capacity(count as usize);
                $func($($arg,)* &mut count, data.as_mut_ptr());
                data.set_len(count as usize);
                data
            }
        }
    };
}

macro_rules! vk_create {
    ($func:expr, $( $arg:expr ),*) => {
        {
            let mut value = Default::default();
            unsafe { vk_assert!($func($($arg,)* &mut value)); }
            value
        }
    };
}

macro_rules! vk_assert {
    ($e:expr) => {
        assert_eq!($e, vk::VkResult::VK_SUCCESS)
    };
}

// --- VULKAN CONTEXT SETUP ---

unsafe extern "C" fn my_glfw_error_callback(error: i32, description:
*const std::ffi::c_char) {
    panic!("GLFW Error {}: {}", error, unsafe {
        std::ffi::CStr::from_ptr(description).to_str().unwrap()
```

```
    });
}

struct GlfwContext<'a>(&'a mut vk::GLFWwindow);
impl<'a> GlfwContext<'a> {
    fn new() -> Self {
        unsafe
{ vk::glfwSetErrorCallback(Some(my_glfw_error_callback)) };
        assert_eq!(unsafe { vk::glfwInit() }, vk::GLFW_TRUE as i32);
        unsafe {
            vk::glfwWindowHint(vk::GLFW_CLIENT_API as i32,
vk::GLFW_NO_API as i32);
        }
        unsafe {
            vk::glfwWindowHint(vk::GLFW_RESIZABLE as i32,
vk::GLFW_FALSE as i32);
        }
        let glfw_window = unsafe {
            vk::glfwCreateWindow(
                GRID_WIDTH as i32,
                GRID_HEIGHT as i32,
                c_str!("Brownian Demo"),
                std::ptr::null_mut(),
                std::ptr::null_mut(),
            )
        };
        Self(unsafe
{ std::ptr::NonNull::new(glfw_window).unwrap().as_mut() })
    }
}
impl<'a> Drop for GlfwContext<'a> {
    fn drop(&mut self) {
        unsafe {
            vk::glfwDestroyWindow(self.0);
            vk::glfwTerminate();
        }
    }
}

fn create_instance() -> vk::VkInstance {
    let vk_application_info = vk::VkApplicationInfo {
        sType: vk::VkStructureType::VK_STRUCTURE_TYPE_APPLICATION_INFO,
        pApplicationName: c_str!("Demo"),
        applicationVersion: vk::vk_make_version(1u32, 0u32, 0u32),
        pEngineName: c_str!("Demo Engine"),
        engineVersion: vk::vk_make_version(1u32, 0u32, 0u32),
        apiVersion: vk::vk_make_api_version(0, 1, 3, 0),
        ..Default::default()
```

```
    };

    let mut enabled_extension_names = vec![
        c_str!("VK_EXT_debug_utils"),
        c_str!("VK_KHR_get_physical_device_properties2"),
        c_str!("VK_KHR_external_memory_capabilities"),
    ];

    let mut glfw_required_instance_extensions_count: u32 = 0;
    let glfw_required_instance_extensions = unsafe
{ vk::glfwGetRequiredInstanceExtensions(&mut
glfw_required_instance_extensions_count) };
    let glfw_required_instance_extensions =
        unsafe
{ std::slice::from_raw_parts(glfw_required_instance_extensions,
glfw_required_instance_extensions_count as usize) };
enabled_extension_names.extend(glfw_required_instance_extensions.iter());

    let enabled_layer_names = [c_str!("VK_LAYER_KHRONOS_validation")];

    let vk_instance_create_info = vk::VkInstanceCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO,
        pApplicationInfo: &vk_application_info,
        ppEnabledExtensionNames: enabled_extension_names.as_ptr() as
*const *const _,
        enabledExtensionCount: enabled_extension_names.len() as u32,
        ppEnabledLayerNames: enabled_layer_names.as_ptr() as *const
*const _,
        enabledLayerCount: enabled_layer_names.len() as u32,
        ..Default::default()
    };
    let mut vk_instance: vk::VkInstance = std::ptr::null_mut();
    unsafe {
        vk_assert!(vk::vkCreateInstance(&vk_instance_create_info,
std::ptr::null(), &mut vk_instance));
    }
    return vk_instance;
}

fn pick_physical_device(vk_instance: vk::VkInstance) ->
vk::VkPhysicalDevice {
    let physical_devices = vk_enumerate!
(vk::vkEnumeratePhysicalDevices, vk_instance);
    for physical_device in physical_devices {
        let mut props: vk::VkPhysicalDeviceProperties =
Default::default();
```

```
        unsafe {
            vk::vkGetPhysicalDeviceProperties(physical_device, &mut
props);
        }
        if props.deviceType ==
vk::VkPhysicalDeviceType::VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU
            || props.deviceType ==
vk::VkPhysicalDeviceType::VK_PHYSICAL_DEVICE_TYPE_INTEGRATED_GPU
        {
            return physical_device;
        }
    }
    std::ptr::null_mut()
}

fn pick_queue_family(vk_physical_device: vk::VkPhysicalDevice,
vk_surface_khr: vk::VkSurfaceKHR) -> u32 {
    let props = vk_enumerate!
(vk::vkGetPhysicalDeviceQueueFamilyProperties, vk_physical_device);
    props
        .iter()
        .enumerate()
        .find(|(index, prop)| {
            let mut supported = vk::VK_FALSE;
            unsafe {
                vk_assert!(vk::vkGetPhysicalDeviceSurfaceSupportKHR(
                    vk_physical_device,
                    *index as u32,
                    vk_surface_khr,
                    &mut supported
                ));
            }
            supported == vk::VK_TRUE
                && contains_bits!(prop.queueFlags,
vk::VkQueueFlagBits::VK_QUEUE_GRAPHICS_BIT)
                && contains_bits!(prop.queueFlags,
vk::VkQueueFlagBits::VK_QUEUE_COMPUTE_BIT)
        })
        .unwrap()
        .0 as u32
}

fn create_device(vk_physical_device: vk::VkPhysicalDevice,
queue_family_index: u32) -> vk::VkDevice {
    let required_extensions = [
        c_str!("VK_KHR_swapchain"),
        c_str!("VK_KHR_dynamic_rendering"),
        // VK_EXT_shader_object REMOVED
```

```
    ];

    let mut vk_13_features = vk::VkPhysicalDeviceVulkan13Features {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_3_FEATURES,
        dynamicRendering: vk::VK_TRUE,
        synchronization2: vk::VK_TRUE,
        ..Default::default()
    };

    let mut vk_12_features = vk::VkPhysicalDeviceVulkan12Features {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_2_FEATURES,
        vulkanMemoryModel: vk::VK_TRUE,
        vulkanMemoryModelDeviceScope: vk::VK_TRUE,
        pNext: &mut vk_13_features as *mut _ as *mut std::ffi::c_void,
        ..Default::default()
    };

    let queue_priority: f32 = 1.0;
    let queue_info = vk::VkDeviceQueueCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO,
        queueFamilyIndex: queue_family_index,
        queueCount: 1,
        pQueuePriorities: &queue_priority,
        ..Default::default()
    };

    let create_info = vk::VkDeviceCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO,
        pNext: &mut vk_12_features as *mut _ as *mut std::ffi::c_void,
        pQueueCreateInfos: &queue_info,
        queueCreateInfoCount: 1,
        ppEnabledExtensionNames: required_extensions.as_ptr(),
        enabledExtensionCount: required_extensions.len() as u32,
        ..Default::default()
    };

    let mut vk_device: vk::VkDevice = Default::default();
    unsafe {
        vk_assert!(vk::vkCreateDevice(vk_physical_device, &create_info,
std::ptr::null(), &mut vk_device));
    }
    vk_device
}
```

```
// --- MEMORY HELPERS ---

fn find_memory_type_index(vk_physical_device: vk::VkPhysicalDevice,
memory_type_bits: u32, property_flags: vk::VkMemoryPropertyFlagBits) ->
u32 {
    let mut props = vk::VkPhysicalDeviceMemoryProperties::default();
    unsafe {
        vk::vkGetPhysicalDeviceMemoryProperties(vk_physical_device,
&mut props);
    }
    for i in 0..props.memoryTypeCount {
        if (memory_type_bits & (1 << i)) != 0 && (props.memoryTypes[i
as usize].propertyFlags & property_flags).0 == property_flags.0 {
            return i;
        }
    }
    panic!("Failed to find suitable memory type.");
}

// Just load the dynamic rendering commands now
struct DynamicRenderingFns {
    vkCmdBeginRenderingKHR: unsafe extern "system"
fn(vk::VkCommandBuffer, *const vk::VkRenderingInfoKHR),
    vkCmdEndRenderingKHR: unsafe extern "system"
fn(vk::VkCommandBuffer),
}

impl DynamicRenderingFns {
    fn new(device: vk::VkDevice) -> Self {
        unsafe {
            Self {
                vkCmdBeginRenderingKHR:
std::mem::transmute(vk::vkGetDeviceProcAddr(device, c_str!
("vkCmdBeginRenderingKHR")).unwrap()),
                vkCmdEndRenderingKHR:
std::mem::transmute(vk::vkGetDeviceProcAddr(device, c_str!
("vkCmdEndRenderingKHR")).unwrap()),
            }
        }
    }
}

// --- MAIN APPLICATION ---

macro_rules! include_spirv {
    ($path:expr) => {{
        const BYTES: &[u8] = include_bytes!(env!($path));
        const _: () = assert!(BYTES.len() % 4 == 0, "SPIR-V binary must
```

```
be 32-bit aligned");
        #[repr(C)]
        union Transmuter {
            bytes: [u8; BYTES.len()],
            words: [u32; BYTES.len() / 4],
        }
        const TRANSMUTED: &[u32] = unsafe {
            &Transmuter {
                bytes: *include_bytes!(env!($path)),
            }
            .words
        };
        TRANSMUTED
    }};
}

const SHADER_CODE: &[u32] = include_spirv!("SHADERS_PATH");

const REQUIRED_SURFACE_FORMAT: vk::VkSurfaceFormatKHR =
vk::VkSurfaceFormatKHR {
    format: vk::VkFormat::VK_FORMAT_B8G8R8A8_UNORM,
    colorSpace: vk::VkColorSpaceKHR::VK_COLOR_SPACE_SRGB_NONLINEAR_KHR,
};
const REQUIRED_PRESENT_MODE: vk::VkPresentModeKHR =
vk::VkPresentModeKHR::VK_PRESENT_MODE_MAILBOX_KHR;

fn recreate_swapchain(
    vk_swapchain_khr: &mut vk::VkSwapchainKHR,
    glfw_window: *mut vk::GLFWwindow,
    vk_physical_device: vk::VkPhysicalDevice,
    vk_surface_khr: vk::VkSurfaceKHR,
    vk_device: vk::VkDevice,
) -> vk::VkExtent2D {
    let mut vk_surface_capabilities_khr =
vk::VkSurfaceCapabilitiesKHR::default();
    unsafe {
        vk_assert!(vk::vkGetPhysicalDeviceSurfaceCapabilitiesKHR(
            vk_physical_device,
            vk_surface_khr,
            &mut vk_surface_capabilities_khr
        ));
    }

    let vk_surface_format_khrs = vk_enumerate!
(vk::vkGetPhysicalDeviceSurfaceFormatsKHR, vk_physical_device,
vk_surface_khr);
    assert!(
        vk_surface_format_khrs
```

```
            .iter()
            .any(|f| { f.format == REQUIRED_SURFACE_FORMAT.format &&
f.colorSpace == REQUIRED_SURFACE_FORMAT.colorSpace })
    );

    let vk_swapchain_extent = unsafe {
        let mut width: i32 = 0;
        let mut height: i32 = 0;
        vk::glfwGetFramebufferSize(glfw_window, &mut width, &mut
height);
        vk::VkExtent2D {
            width: (width as u32).clamp(
                vk_surface_capabilities_khr.minImageExtent.width,
                vk_surface_capabilities_khr.maxImageExtent.width,
            ),
            height: (height as u32).clamp(
                vk_surface_capabilities_khr.minImageExtent.height,
                vk_surface_capabilities_khr.maxImageExtent.height,
            ),
        }
    };

    let old_swapchain = *vk_swapchain_khr;

    let vk_swapchain_create_info_khr = vk::VkSwapchainCreateInfoKHR {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR,
        surface: vk_surface_khr,
        minImageCount: vk_surface_capabilities_khr.minImageCount + 1,
        imageFormat: REQUIRED_SURFACE_FORMAT.format,
        imageColorSpace: REQUIRED_SURFACE_FORMAT.colorSpace,
        imageExtent: vk_swapchain_extent,
        imageArrayLayers: 1,
        imageUsage:
vk::VkImageUsageFlagBits::VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT |
vk::VkImageUsageFlagBits::VK_IMAGE_USAGE_TRANSFER_DST_BIT,
        imageSharingMode: vk::VkSharingMode::VK_SHARING_MODE_EXCLUSIVE,
        preTransform: vk_surface_capabilities_khr.currentTransform,
        compositeAlpha:
vk::VkCompositeAlphaFlagBitsKHR::VK_COMPOSITE_ALPHA_OPAQUE_BIT_KHR,
        presentMode: REQUIRED_PRESENT_MODE,
        clipped: vk::VK_TRUE,
        oldSwapchain: old_swapchain,
        ..Default::default()
    };

    unsafe {
        vk_assert!(vk::vkCreateSwapchainKHR(
```

```
            vk_device,
            &vk_swapchain_create_info_khr,
            std::ptr::null(),
            vk_swapchain_khr
        ));

        if old_swapchain != std::ptr::null_mut() {
            vk::vkDestroySwapchainKHR(vk_device, old_swapchain,
std::ptr::null());
        }
    }
    vk_swapchain_extent
}

fn main() {
    let mut glfw_context = GlfwContext::new();
    let vk_instance = create_instance();
    defer! { unsafe { vk::vkDestroyInstance(vk_instance,
std::ptr::null()) } }

    let vk_physical_device = pick_physical_device(vk_instance);
    let mut vk_surface_khr: vk::VkSurfaceKHR = std::ptr::null_mut();
    unsafe {
        vk::glfwCreateWindowSurface(
            std::mem::transmute(vk_instance),
            glfw_context.0,
            std::ptr::null(),
            std::mem::transmute(&mut vk_surface_khr),
        );
    }
    defer! { unsafe { vk::vkDestroySurfaceKHR(vk_instance,
vk_surface_khr, std::ptr::null()); } }

    let queue_family_index = pick_queue_family(vk_physical_device,
vk_surface_khr);
    let vk_device = create_device(vk_physical_device,
queue_family_index);
    defer! { unsafe { vk::vkDestroyDevice(vk_device,
std::ptr::null()); } }

    let ext_fns = DynamicRenderingFns::new(vk_device);

    let mut vk_queue: vk::VkQueue = Default::default();
    unsafe {
        vk::vkGetDeviceQueue(vk_device, queue_family_index, 0, &mut
vk_queue);
    }
```

```
    let vk_command_pool = vk_create!(
        vk::vkCreateCommandPool,
        vk_device,
        &vk::VkCommandPoolCreateInfo {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO,
            flags:
vk::VkCommandPoolCreateFlagBits::VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_B
            queueFamilyIndex: queue_family_index,
            ..Default::default()
        },
        std::ptr::null()
    );
    defer! { unsafe { vk::vkDestroyCommandPool(vk_device,
vk_command_pool, std::ptr::null()) } }

    let mut vk_command_buffer = vk::VkCommandBuffer::default();
    unsafe {
        vk_assert!(vk::vkAllocateCommandBuffers(
            vk_device,
            &vk::VkCommandBufferAllocateInfo {
                sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO,
                commandPool: vk_command_pool,
                level:
vk::VkCommandBufferLevel::VK_COMMAND_BUFFER_LEVEL_PRIMARY,
                commandBufferCount: 1,
                ..Default::default()
            },
            &mut vk_command_buffer
        ));
    }

    // --- BUFFER CREATION (PARTICLES & GRID) ---
    let particles_size = (NUM_PARTICLES as usize *
std::mem::size_of::<Particle>()) as u64;
    let grid_size = (GRID_WIDTH as usize * GRID_HEIGHT as usize *
std::mem::size_of::<u32>()) as u64;

    let create_buffer = |size, usage| {
        let info = vk::VkBufferCreateInfo {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO,
            size,
            usage,
            sharingMode: vk::VkSharingMode::VK_SHARING_MODE_EXCLUSIVE,
            ..Default::default()
        };
```

```
        vk_create!(vk::vkCreateBuffer, vk_device, &info,
std::ptr::null())
    };

    let vk_staging_buffer = create_buffer(particles_size + grid_size,
vk::VkBufferUsageFlagBits::VK_BUFFER_USAGE_TRANSFER_SRC_BIT);
    let vk_particle_buffer = create_buffer(
        particles_size,
        vk::VkBufferUsageFlagBits::VK_BUFFER_USAGE_STORAGE_BUFFER_BIT |
vk::VkBufferUsageFlagBits::VK_BUFFER_USAGE_TRANSFER_DST_BIT,
    );
    let vk_grid_buffer = create_buffer(
        grid_size,
        vk::VkBufferUsageFlagBits::VK_BUFFER_USAGE_STORAGE_BUFFER_BIT
            |
vk::VkBufferUsageFlagBits::VK_BUFFER_USAGE_TRANSFER_DST_BIT
            |
vk::VkBufferUsageFlagBits::VK_BUFFER_USAGE_TRANSFER_SRC_BIT, // Added
SRC_BIT
    );

    let vk_readback_buffer = create_buffer(grid_size,
vk::VkBufferUsageFlagBits::VK_BUFFER_USAGE_TRANSFER_DST_BIT);

    let bind_memory = |buffer: vk::VkBuffer, flags| {
        let mut reqs = vk::VkMemoryRequirements::default();
        unsafe {
            vk::vkGetBufferMemoryRequirements(vk_device, buffer, &mut
reqs);
        }
        let alloc_info = vk::VkMemoryAllocateInfo {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO,
            allocationSize: reqs.size,
            memoryTypeIndex: find_memory_type_index(vk_physical_device,
reqs.memoryTypeBits, flags),
            ..Default::default()
        };
        let mem = vk_create!(vk::vkAllocateMemory, vk_device,
&alloc_info, std::ptr::null());
        unsafe {
            vk::vkBindBufferMemory(vk_device, buffer, mem, 0);
        }
        mem
    };

    let staging_mem = bind_memory(
        vk_staging_buffer,
```

```
vk::VkMemoryPropertyFlagBits::VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
vk::VkMemoryPropertyFlagBits::VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
    );
    let particle_mem = bind_memory(vk_particle_buffer,
vk::VkMemoryPropertyFlagBits::VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT);
    let grid_mem = bind_memory(vk_grid_buffer,
vk::VkMemoryPropertyFlagBits::VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT);

    let readback_mem = bind_memory(
        vk_readback_buffer,
vk::VkMemoryPropertyFlagBits::VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
vk::VkMemoryPropertyFlagBits::VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
    );

    let mut readback_ptr = std::ptr::null_mut();
    unsafe {
        vk_assert!(vk::vkMapMemory(
            vk_device,
            readback_mem,
            0,
            grid_size,
            vk::VkMemoryMapFlagBits(0),
            &mut readback_ptr
        ));
    }

    unsafe {
        let mut mapped_ptr = std::ptr::null_mut();
        vk::vkMapMemory(
            vk_device,
            staging_mem,
            0,
            particles_size + grid_size,
            vk::VkMemoryMapFlagBits(0),
            &mut mapped_ptr,
        );
        let staging_particles =
std::slice::from_raw_parts_mut(mapped_ptr as *mut Particle,
NUM_PARTICLES as usize);
        for i in 0..NUM_PARTICLES as usize {
            let mut rng = Xorshift32 { state: 1337 + i as u32 };
            staging_particles[i] = Particle {
                pos_x: rng.next() % GRID_WIDTH,
                pos_y: rng.next() % GRID_HEIGHT,
                rng,
                _padding: 0,
            };
```

```
        }
        let staging_grid =
std::slice::from_raw_parts_mut(mapped_ptr.add(particles_size as usize)
as *mut u32, (GRID_WIDTH * GRID_HEIGHT) as usize);
        staging_grid.fill(0);
        vk::vkUnmapMemory(vk_device, staging_mem);

        let begin_info = vk::VkCommandBufferBeginInfo {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO,
            flags:
vk::VkCommandBufferUsageFlagBits::VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT,
            ..Default::default()
        };
        vk::vkBeginCommandBuffer(vk_command_buffer, &begin_info);
        vk::vkCmdCopyBuffer(
            vk_command_buffer,
            vk_staging_buffer,
            vk_particle_buffer,
            1,
            &vk::VkBufferCopy {
                srcOffset: 0,
                dstOffset: 0,
                size: particles_size,
            },
        );
        vk::vkCmdCopyBuffer(
            vk_command_buffer,
            vk_staging_buffer,
            vk_grid_buffer,
            1,
            &vk::VkBufferCopy {
                srcOffset: particles_size,
                dstOffset: 0,
                size: grid_size,
            },
        );
        vk::vkEndCommandBuffer(vk_command_buffer);

        let submit = vk::VkSubmitInfo {
            sType: vk::VkStructureType::VK_STRUCTURE_TYPE_SUBMIT_INFO,
            commandBufferCount: 1,
            pCommandBuffers: &vk_command_buffer,
            ..Default::default()
        };
        vk::vkQueueSubmit(vk_queue, 1, &submit, std::ptr::null_mut());
        vk::vkQueueWaitIdle(vk_queue);
    }
```

```
    // --- DESCRIPTOR SETS ---
    let layout_bindings = [
        vk::VkDescriptorSetLayoutBinding {
            binding: 0,
            descriptorType:
vk::VkDescriptorType::VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
            descriptorCount: 1,
            stageFlags:
vk::VkShaderStageFlagBits::VK_SHADER_STAGE_COMPUTE_BIT,
            ..Default::default()
        },
        vk::VkDescriptorSetLayoutBinding {
            binding: 1,
            descriptorType:
vk::VkDescriptorType::VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
            descriptorCount: 1,
            stageFlags: vk::VkShaderStageFlagBits(

vk::VkShaderStageFlagBits::VK_SHADER_STAGE_COMPUTE_BIT.0 |
vk::VkShaderStageFlagBits::VK_SHADER_STAGE_FRAGMENT_BIT.0,
            ),
            ..Default::default()
        },
    ];
    let layout_info = vk::VkDescriptorSetLayoutCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO,
        bindingCount: 2,
        pBindings: layout_bindings.as_ptr(),
        ..Default::default()
    };
    let desc_layout = vk_create!(vk::vkCreateDescriptorSetLayout,
vk_device, &layout_info, std::ptr::null());

    let pool_sizes = [vk::VkDescriptorPoolSize {
        type_: vk::VkDescriptorType::VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
        descriptorCount: 2,
    }];
    let desc_pool = vk_create!(
        vk::vkCreateDescriptorPool,
        vk_device,
        &vk::VkDescriptorPoolCreateInfo {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO,
            poolSizeCount: 1,
            pPoolSizes: pool_sizes.as_ptr(),
            maxSets: 1,
```

```
            ..Default::default()
        },
        std::ptr::null()
    );

    let mut desc_set = Default::default();
    unsafe {
        vk_assert!(vk::vkAllocateDescriptorSets(
            vk_device,
            &vk::VkDescriptorSetAllocateInfo {
                sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO,
                descriptorPool: desc_pool,
                descriptorSetCount: 1,
                pSetLayouts: &desc_layout,
                ..Default::default()
            },
            &mut desc_set
        ));
    }

    let p_info = vk::VkDescriptorBufferInfo {
        buffer: vk_particle_buffer,
        offset: 0,
        range: vk::VK_WHOLE_SIZE,
    };
    let g_info = vk::VkDescriptorBufferInfo {
        buffer: vk_grid_buffer,
        offset: 0,
        range: vk::VK_WHOLE_SIZE,
    };
    let writes = [
        vk::VkWriteDescriptorSet {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET,
            dstSet: desc_set,
            dstBinding: 0,
            descriptorCount: 1,
            descriptorType:
vk::VkDescriptorType::VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
            pBufferInfo: &p_info,
            ..Default::default()
        },
        vk::VkWriteDescriptorSet {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET,
            dstSet: desc_set,
            dstBinding: 1,
```

```
            descriptorCount: 1,
            descriptorType:
vk::VkDescriptorType::VK_DESCRIPTOR_TYPE_STORAGE_BUFFER,
            pBufferInfo: &g_info,
            ..Default::default()
        },
    ];
    unsafe {
        vk::vkUpdateDescriptorSets(vk_device, 2, writes.as_ptr(), 0,
std::ptr::null());
    }

    let push_range = vk::VkPushConstantRange {
        stageFlags: vk::VkShaderStageFlagBits(
            vk::VkShaderStageFlagBits::VK_SHADER_STAGE_COMPUTE_BIT.0 |
vk::VkShaderStageFlagBits::VK_SHADER_STAGE_FRAGMENT_BIT.0,
        ),
        offset: 0,
        size: std::mem::size_of::<PushConstants>() as u32,
    };
    let pipeline_layout = vk_create!(
        vk::vkCreatePipelineLayout,
        vk_device,
        &vk::VkPipelineLayoutCreateInfo {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO,
            setLayoutCount: 1,
            pSetLayouts: &desc_layout,
            pushConstantRangeCount: 1,
            pPushConstantRanges: &push_range,
            ..Default::default()
        },
        std::ptr::null()
    );

    // --- SHADER MODULE CREATION ---
    let shader_module_info = vk::VkShaderModuleCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO,
        codeSize: SHADER_CODE.len() * 4,
        pCode: SHADER_CODE.as_ptr() as *const _,
        ..Default::default()
    };
    let shader_module = vk_create!(vk::vkCreateShaderModule, vk_device,
&shader_module_info, std::ptr::null());
    defer! { unsafe { vk::vkDestroyShaderModule(vk_device,
shader_module, std::ptr::null()); } }
```

```
    // --- PIPELINE CREATION ---

    // 1. Clean Compute Pipeline
    let clean_stage = vk::VkPipelineShaderStageCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO,
        stage: vk::VkShaderStageFlagBits::VK_SHADER_STAGE_COMPUTE_BIT,
        module: shader_module,
        pName: c_str!("clean_buffer"),
        ..Default::default()
    };
    let clean_pipeline_info = vk::VkComputePipelineCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO,
        stage: clean_stage,
        layout: pipeline_layout,
        ..Default::default()
    };
    let mut clean_pipeline = Default::default();
    unsafe {
        vk_assert!(vk::vkCreateComputePipelines(
            vk_device,
            std::ptr::null_mut(),
            1,
            &clean_pipeline_info,
            std::ptr::null(),
            &mut clean_pipeline
        ));
    }
    defer! { unsafe { vk::vkDestroyPipeline(vk_device, clean_pipeline,
std::ptr::null()); } }

    // 2. Step Compute Pipeline
    let step_stage = vk::VkPipelineShaderStageCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO,
        stage: vk::VkShaderStageFlagBits::VK_SHADER_STAGE_COMPUTE_BIT,
        module: shader_module,
        pName: c_str!("brownian_step"),
        ..Default::default()
    };
    let step_pipeline_info = vk::VkComputePipelineCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_COMPUTE_PIPELINE_CREATE_INFO,
        stage: step_stage,
        layout: pipeline_layout,
        ..Default::default()
    };
```

```
    let mut step_pipeline = Default::default();
    unsafe {
        vk_assert!(vk::vkCreateComputePipelines(
            vk_device,
            std::ptr::null_mut(),
            1,
            &step_pipeline_info,
            std::ptr::null(),
            &mut step_pipeline
        ));
    }
    defer! { unsafe { vk::vkDestroyPipeline(vk_device, step_pipeline,
std::ptr::null()); } }

    // 3. Graphics Pipeline
    let graphics_stages = [
        vk::VkPipelineShaderStageCreateInfo {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO,
            stage:
vk::VkShaderStageFlagBits::VK_SHADER_STAGE_VERTEX_BIT,
            module: shader_module,
            pName: c_str!("fullscreen_vs"),
            ..Default::default()
        },
        vk::VkPipelineShaderStageCreateInfo {
            sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO,
            stage:
vk::VkShaderStageFlagBits::VK_SHADER_STAGE_FRAGMENT_BIT,
            module: shader_module,
            pName: c_str!("grid_fs"),
            ..Default::default()
        },
    ];

    let vertex_input_info = vk::VkPipelineVertexInputStateCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO
        ..Default::default()
    };
    let input_assembly_info =
vk::VkPipelineInputAssemblyStateCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_IN
        topology:
vk::VkPrimitiveTopology::VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST,
        ..Default::default()
```

```
    };
    let viewport_state_info = vk::VkPipelineViewportStateCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO,
        viewportCount: 1,
        scissorCount: 1,
        ..Default::default()
    };
    let rasterizer_info = vk::VkPipelineRasterizationStateCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INF
        polygonMode: vk::VkPolygonMode::VK_POLYGON_MODE_FILL,
        lineWidth: 1.0,
        cullMode: vk::VkCullModeFlagBits::VK_CULL_MODE_NONE,
        frontFace: vk::VkFrontFace::VK_FRONT_FACE_CLOCKWISE,
        ..Default::default()
    };
    let multisampling_info = vk::VkPipelineMultisampleStateCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO,
        rasterizationSamples:
vk::VkSampleCountFlagBits::VK_SAMPLE_COUNT_1_BIT,
        ..Default::default()
    };
    let color_blend_attachment =
vk::VkPipelineColorBlendAttachmentState {
        colorWriteMask:
vk::VkColorComponentFlagBits::VK_COLOR_COMPONENT_R_BIT
            | vk::VkColorComponentFlagBits::VK_COLOR_COMPONENT_G_BIT
            | vk::VkColorComponentFlagBits::VK_COLOR_COMPONENT_B_BIT
            | vk::VkColorComponentFlagBits::VK_COLOR_COMPONENT_A_BIT,
        blendEnable: vk::VK_FALSE,
        ..Default::default()
    };
    let color_blending_info = vk::VkPipelineColorBlendStateCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO,
        attachmentCount: 1,
        pAttachments: &color_blend_attachment,
        ..Default::default()
    };
    let dynamic_states = [
        vk::VkDynamicState::VK_DYNAMIC_STATE_VIEWPORT,
        vk::VkDynamicState::VK_DYNAMIC_STATE_SCISSOR,
    ];
    let dynamic_state_info = vk::VkPipelineDynamicStateCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO,
```

```
        dynamicStateCount: 2,
        pDynamicStates: dynamic_states.as_ptr(),
        ..Default::default()
    };
    let mut pipeline_rendering_info =
vk::VkPipelineRenderingCreateInfoKHR {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PIPELINE_RENDERING_CREATE_INFO_KHR,
        colorAttachmentCount: 1,
        pColorAttachmentFormats: &REQUIRED_SURFACE_FORMAT.format,
        ..Default::default()
    };
    let graphics_pipeline_info = vk::VkGraphicsPipelineCreateInfo {
        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_GRAPHICS_PIPELINE_CREATE_INFO,
        pNext: &mut pipeline_rendering_info as *mut _ as *mut
std::ffi::c_void,
        stageCount: 2,
        pStages: graphics_stages.as_ptr(),
        pVertexInputState: &vertex_input_info,
        pInputAssemblyState: &input_assembly_info,
        pViewportState: &viewport_state_info,
        pRasterizationState: &rasterizer_info,
        pMultisampleState: &multisampling_info,
        pColorBlendState: &color_blending_info,
        pDynamicState: &dynamic_state_info,
        layout: pipeline_layout,
        ..Default::default()
    };

    let mut graphics_pipeline = Default::default();
    unsafe {
        vk_assert!(vk::vkCreateGraphicsPipelines(
            vk_device,
            std::ptr::null_mut(),
            1,
            &graphics_pipeline_info,
            std::ptr::null(),
            &mut graphics_pipeline
        ));
    }
    defer! { unsafe { vk::vkDestroyPipeline(vk_device,
graphics_pipeline, std::ptr::null()); } }

    let mut vk_swapchain_khr: vk::VkSwapchainKHR =
std::ptr::null_mut();

    'swapchain_loop: loop {
```

```
        let vk_present_complete_semaphore = vk_create!(
            vk::vkCreateSemaphore,
            vk_device,
            &vk::VkSemaphoreCreateInfo {
                sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO,
                ..Default::default()
            },
            std::ptr::null()
        );
        defer! { unsafe { vk::vkDestroySemaphore(vk_device,
vk_present_complete_semaphore, std::ptr::null()); } }

        let mut vk_fence = Default::default();
        unsafe {
            vk_assert!(vk::vkCreateFence(
                vk_device,
                &vk::VkFenceCreateInfo {
                    sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_FENCE_CREATE_INFO,
                    flags:
vk::VkFenceCreateFlagBits::VK_FENCE_CREATE_SIGNALED_BIT,
                    ..Default::default()
                },
                std::ptr::null(),
                &mut vk_fence
            ));
        }
        defer! { unsafe { vk::vkDestroyFence(vk_device, vk_fence,
std::ptr::null()); } }

        let vk_swapchain_extent = recreate_swapchain(&mut
vk_swapchain_khr, glfw_context.0, vk_physical_device, vk_surface_khr,
vk_device);
        let vk_swapchain_images = vk_enumerate!
(vk::vkGetSwapchainImagesKHR, vk_device, vk_swapchain_khr);

        let vk_swapchain_image_views: Vec<vk::VkImageView> =
vk_swapchain_images
            .iter()
            .map(|&image| {
                let info = vk::VkImageViewCreateInfo {
                    sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO,
                    image,
                    viewType:
vk::VkImageViewType::VK_IMAGE_VIEW_TYPE_2D,
                    format: REQUIRED_SURFACE_FORMAT.format,
```

```
                    components: vk::VkComponentMapping {
                        r:
vk::VkComponentSwizzle::VK_COMPONENT_SWIZZLE_IDENTITY,
                        g:
vk::VkComponentSwizzle::VK_COMPONENT_SWIZZLE_IDENTITY,
                        b:
vk::VkComponentSwizzle::VK_COMPONENT_SWIZZLE_IDENTITY,
                        a:
vk::VkComponentSwizzle::VK_COMPONENT_SWIZZLE_IDENTITY,
                    },
                    subresourceRange: vk::VkImageSubresourceRange {
                        aspectMask:
vk::VkImageAspectFlagBits::VK_IMAGE_ASPECT_COLOR_BIT,
                        baseMipLevel: 0,
                        levelCount: 1,
                        baseArrayLayer: 0,
                        layerCount: 1,
                    },
                    ..Default::default()
                };
                vk_create!(vk::vkCreateImageView, vk_device, &info,
std::ptr::null())
            })
            .collect();
        defer! { for &view in &vk_swapchain_image_views { unsafe
{ vk::vkDestroyImageView(vk_device, view, std::ptr::null()); } } }

        let vk_render_complete_semaphores: Vec<vk::VkSemaphore> =
(0..vk_swapchain_images.len())
            .map(|_| {
                vk_create!(
                    vk::vkCreateSemaphore,
                    vk_device,
                    &vk::VkSemaphoreCreateInfo {
                        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO,
                        ..Default::default()
                    },
                    std::ptr::null()
                )
            })
            .collect();
        defer! { for &sem in &vk_render_complete_semaphores { unsafe
{ vk::vkDestroySemaphore(vk_device, sem, std::ptr::null()); } } }

        defer! { unsafe { vk_assert!
(vk::vkDeviceWaitIdle(vk_device)); } }
```

```
        let mut frame_count = 0;
        let mut readback_pending = false;

        // --- THE MAIN RENDER LOOP ---
        'render_loop: loop {
            if unsafe { vk::glfwWindowShouldClose(glfw_context.0) } ==
vk::GLFW_TRUE as i32 {
                break 'swapchain_loop;
            }
            unsafe {
                vk::glfwPollEvents();
            }

            unsafe {
                vk_assert!(vk::vkWaitForFences(vk_device, 1, &vk_fence,
vk::VK_TRUE, u64::MAX));
                vk_assert!(vk::vkResetFences(vk_device, 1, &vk_fence));
            }

            if readback_pending {
                let readback_grid = unsafe
{ std::slice::from_raw_parts(readback_ptr as *const u32, (GRID_WIDTH *
GRID_HEIGHT) as usize) };
                let total_particles: u32 = readback_grid.iter().sum();

                println!("Frame {:04}: Total Particles = {}",
frame_count, total_particles);
                assert_eq!(total_particles, NUM_PARTICLES, "PARTICLE
CONSERVATION FAILED!");

                readback_pending = false;
            }
            frame_count += 1;

            let mut image_index: u32 = 0;
            let vk_result = unsafe {
                vk::vkAcquireNextImageKHR(
                    vk_device,
                    vk_swapchain_khr,
                    u64::MAX,
                    vk_present_complete_semaphore,
                    std::ptr::null_mut(),
                    &mut image_index,
                )
            };
            match vk_result {
                vk::VkResult::VK_ERROR_OUT_OF_DATE_KHR |
vk::VkResult::VK_SUBOPTIMAL_KHR => continue 'swapchain_loop,
```

```
                other => vk_assert!(other),
            }

            unsafe {
                vk_assert!(vk::vkBeginCommandBuffer(
                    vk_command_buffer,
                    &vk::VkCommandBufferBeginInfo {
                        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO,
                        flags:
vk::VkCommandBufferUsageFlagBits::VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT,
                        ..Default::default()
                    }
                ));

                let push = PushConstants {
                    grid_width: GRID_WIDTH,
                    grid_height: GRID_HEIGHT,
                    num_particles: NUM_PARTICLES,
                    max_density: MAX_DENSITY,
                };
                vk::vkCmdPushConstants(
                    vk_command_buffer,
                    pipeline_layout,
                    vk::VkShaderStageFlagBits(
vk::VkShaderStageFlagBits::VK_SHADER_STAGE_COMPUTE_BIT.0 |
vk::VkShaderStageFlagBits::VK_SHADER_STAGE_FRAGMENT_BIT.0,
                    ),
                    0,
                    std::mem::size_of::<PushConstants>() as u32,
                    &push as *const _ as *const _,
                );

                // --- 1. COMPUTE: CLEAN PASS ---
                vk::vkCmdBindPipeline(vk_command_buffer,
vk::VkPipelineBindPoint::VK_PIPELINE_BIND_POINT_COMPUTE,
clean_pipeline);
                vk::vkCmdBindDescriptorSets(
                    vk_command_buffer,
vk::VkPipelineBindPoint::VK_PIPELINE_BIND_POINT_COMPUTE,
                    pipeline_layout,
                    0,
                    1,
                    &desc_set,
                    0,
                    std::ptr::null(),
                );
```

```
                vk::vkCmdDispatch(vk_command_buffer, ((GRID_WIDTH *
GRID_HEIGHT) + 255) / 256, 1, 1);

                // Barrier: Clean -> Step
                let barrier1 = vk::VkBufferMemoryBarrier {
                    sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER,
                    srcAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_SHADER_WRITE_BIT,
                    dstAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_SHADER_READ_BIT |
vk::VkAccessFlagBits::VK_ACCESS_SHADER_WRITE_BIT,
                    buffer: vk_grid_buffer,
                    size: vk::VK_WHOLE_SIZE,
                    ..Default::default()
                };

                let particle_barrier = vk::VkBufferMemoryBarrier {
                    sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER,
                    srcAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_TRANSFER_WRITE_BIT |
vk::VkAccessFlagBits::VK_ACCESS_SHADER_WRITE_BIT,
                    dstAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_SHADER_READ_BIT |
vk::VkAccessFlagBits::VK_ACCESS_SHADER_WRITE_BIT,
                    buffer: vk_particle_buffer,
                    size: vk::VK_WHOLE_SIZE,
                    ..Default::default()
                };

                let barriers = [barrier1, particle_barrier];
                vk::vkCmdPipelineBarrier(
                    vk_command_buffer,
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_TRANSFER_BIT |
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT,
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT,
                    vk::VkDependencyFlagBits(0),
                    0,
                    std::ptr::null(),
                    2,
                    barriers.as_ptr(),
                    0,
                    std::ptr::null(),
                );

                vk::vkCmdBindPipeline(vk_command_buffer,
```

```
vk::VkPipelineBindPoint::VK_PIPELINE_BIND_POINT_COMPUTE,
step_pipeline);
                vk::vkCmdDispatch(vk_command_buffer, (NUM_PARTICLES +
255) / 256, 1, 1);

                if frame_count % 100 == 0 {
                    let transfer_barrier = vk::VkBufferMemoryBarrier {
                        sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER,
                        srcAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_SHADER_WRITE_BIT,
                        dstAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_TRANSFER_READ_BIT,
                        buffer: vk_grid_buffer,
                        size: vk::VK_WHOLE_SIZE,
                        ..Default::default()
                    };
                    vk::vkCmdPipelineBarrier(
                        vk_command_buffer,
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT,
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_TRANSFER_BIT,
                        vk::VkDependencyFlagBits(0),
                        0,
                        std::ptr::null(),
                        1,
                        &transfer_barrier,
                        0,
                        std::ptr::null(),
                    );

                    let copy_region = vk::VkBufferCopy {
                        srcOffset: 0,
                        dstOffset: 0,
                        size: grid_size,
                    };
                    vk::vkCmdCopyBuffer(vk_command_buffer,
vk_grid_buffer, vk_readback_buffer, 1, &copy_region);
                    readback_pending = true;
                }

                let barrier2 = vk::VkBufferMemoryBarrier {
                    sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER,
                    srcAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_SHADER_WRITE_BIT,
                    dstAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_SHADER_READ_BIT,
```

```
                    buffer: vk_grid_buffer,
                    size: vk::VK_WHOLE_SIZE,
                    ..Default::default()
                };

                let image_to_render_barrier = vk::VkImageMemoryBarrier
{
                    sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER,
                    oldLayout:
vk::VkImageLayout::VK_IMAGE_LAYOUT_UNDEFINED,
                    newLayout:
vk::VkImageLayout::VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
                    srcAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_NONE,
                    dstAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT,
                    image: vk_swapchain_images[image_index as usize],
                    subresourceRange: vk::VkImageSubresourceRange {
                        aspectMask:
vk::VkImageAspectFlagBits::VK_IMAGE_ASPECT_COLOR_BIT,
                        levelCount: 1,
                        layerCount: 1,
                        ..Default::default()
                    },
                    ..Default::default()
                };

                vk::vkCmdPipelineBarrier(
                    vk_command_buffer,
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT
                        |
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT,
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT
                        |
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT,
                    vk::VkDependencyFlagBits(0),
                    0,
                    std::ptr::null(),
                    1,
                    &barrier2,
                    1,
                    &image_to_render_barrier,
                );

                let color_attachment = vk::VkRenderingAttachmentInfoKHR
{
```

```
                    sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO_KHR,
                    imageView: vk_swapchain_image_views[image_index as
usize],
                    imageLayout:
vk::VkImageLayout::VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
                    loadOp:
vk::VkAttachmentLoadOp::VK_ATTACHMENT_LOAD_OP_CLEAR,
                    storeOp:
vk::VkAttachmentStoreOp::VK_ATTACHMENT_STORE_OP_STORE,
                    clearValue: vk::VkClearValue {
                        color: vk::VkClearColorValue {
                            float32: [0.0, 0.0, 0.0, 1.0],
                        },
                    },
                    ..Default::default()
                };

                let rendering_info = vk::VkRenderingInfoKHR {
                    sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_RENDERING_INFO_KHR,
                    renderArea: vk::VkRect2D {
                        offset: vk::VkOffset2D { x: 0, y: 0 },
                        extent: vk_swapchain_extent,
                    },
                    layerCount: 1,
                    colorAttachmentCount: 1,
                    pColorAttachments: &color_attachment,
                    ..Default::default()
                };

                (ext_fns.vkCmdBeginRenderingKHR)(vk_command_buffer,
&rendering_info);

                vk::vkCmdBindPipeline(
                    vk_command_buffer,
vk::VkPipelineBindPoint::VK_PIPELINE_BIND_POINT_GRAPHICS,
                    graphics_pipeline,
                );
                vk::vkCmdBindDescriptorSets(
                    vk_command_buffer,
vk::VkPipelineBindPoint::VK_PIPELINE_BIND_POINT_GRAPHICS,
                    pipeline_layout,
                    0,
                    1,
                    &desc_set,
                    0,
```

```
                    std::ptr::null(),
                );

                let viewport = vk::VkViewport {
                    width: vk_swapchain_extent.width as f32,
                    height: vk_swapchain_extent.height as f32,
                    maxDepth: 1.0,
                    ..Default::default()
                };
                let scissor = vk::VkRect2D {
                    extent: vk_swapchain_extent,
                    ..Default::default()
                };
                vk::vkCmdSetViewport(vk_command_buffer, 0, 1,
&viewport);
                vk::vkCmdSetScissor(vk_command_buffer, 0, 1, &scissor);

                vk::vkCmdDraw(vk_command_buffer, 6, 1, 0, 0);

                (ext_fns.vkCmdEndRenderingKHR)(vk_command_buffer);

                let image_to_present_barrier = vk::VkImageMemoryBarrier
{
                    sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER,
                    oldLayout:
vk::VkImageLayout::VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL,
                    newLayout:
vk::VkImageLayout::VK_IMAGE_LAYOUT_PRESENT_SRC_KHR,
                    srcAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT,
                    dstAccessMask:
vk::VkAccessFlagBits::VK_ACCESS_NONE,
                    image: vk_swapchain_images[image_index as usize],
                    subresourceRange: vk::VkImageSubresourceRange {
                        aspectMask:
vk::VkImageAspectFlagBits::VK_IMAGE_ASPECT_COLOR_BIT,
                        levelCount: 1,
                        layerCount: 1,
                        ..Default::default()
                    },
                    ..Default::default()
                };

                vk::vkCmdPipelineBarrier(
                    vk_command_buffer,
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT,
```

```
vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_BOTTOM_OF_PIPE_BIT,
                    vk::VkDependencyFlagBits(0),
                    0,
                    std::ptr::null(),
                    0,
                    std::ptr::null(),
                    1,
                    &image_to_present_barrier,
                );

                vk_assert!(vk::vkEndCommandBuffer(vk_command_buffer));
            }

            let wait_stages =
[vk::VkPipelineStageFlagBits::VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT];
            let vk_submit_info = vk::VkSubmitInfo {
                sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_SUBMIT_INFO,
                waitSemaphoreCount: 1,
                pWaitSemaphores: &vk_present_complete_semaphore,
                pWaitDstStageMask: wait_stages.as_ptr(),
                commandBufferCount: 1,
                pCommandBuffers: &vk_command_buffer,
                signalSemaphoreCount: 1,
                pSignalSemaphores:
&vk_render_complete_semaphores[image_index as usize],
                ..Default::default()
            };

            unsafe {
                vk_assert!(vk::vkQueueSubmit(vk_queue, 1,
&vk_submit_info, vk_fence));
            }

            let vk_present_info_khr = vk::VkPresentInfoKHR {
                sType:
vk::VkStructureType::VK_STRUCTURE_TYPE_PRESENT_INFO_KHR,
                waitSemaphoreCount: 1,
                pWaitSemaphores:
&vk_render_complete_semaphores[image_index as usize],
                swapchainCount: 1,
                pSwapchains: &vk_swapchain_khr,
                pImageIndices: &image_index,
                ..Default::default()
            };

            match unsafe { vk::vkQueuePresentKHR(vk_queue,
&vk_present_info_khr) } {
```

```
                vk::VkResult::VK_ERROR_OUT_OF_DATE_KHR |
vk::VkResult::VK_SUBOPTIMAL_KHR => continue 'swapchain_loop,
                other => vk_assert!(other),
            }
        }
    }
}
```

A.2 my_shaders/src/lib.rs

```
#![no_std]

use spirv_std::glam::{UVec3, Vec2, Vec4};
use spirv_std::spirv;

#[derive(Clone, Copy)]
pub struct Xorshift32 {
    pub state: u32,
}

impl Xorshift32 {
    pub fn next(&mut self) -> u32 {
        let mut x = self.state;
        x ^= x << 13;
        x ^= x >> 17;
        x ^= x << 5;
        self.state = x;
        x
    }
}

#[repr(C)]
#[derive(Clone, Copy)]
pub struct Particle {
    pub pos_x: u32,
    pub pos_y: u32,
    pub rng: Xorshift32,
    pub _padding: u32,
}

#[repr(C)]
#[derive(Clone, Copy)]
pub struct PushConstants {
    pub grid_width: u32,
    pub grid_height: u32,
    pub num_particles: u32,
    pub max_density: u32,
}
```

```
// --- COMPUTE 1: CLEAN PASS ---
#[spirv(compute(threads(256)))]
pub fn clean_buffer(
    #[spirv(global_invocation_id)] id: UVec3,
    #[spirv(storage_buffer, descriptor_set = 0, binding = 1)] grid:
&mut [u32],
    #[spirv(push_constant)] constants: &PushConstants,
) {
    let idx = id.x as usize;
    if idx < (constants.grid_width * constants.grid_height) as usize {
        grid[idx] = 0;
    }
}

#[spirv(compute(threads(256)))]
pub fn brownian_step(
    #[spirv(global_invocation_id)] id: UVec3,
    #[spirv(storage_buffer, descriptor_set = 0, binding = 0)]
particles: &mut [Particle],
    #[spirv(storage_buffer, descriptor_set = 0, binding = 1)] grid:
&mut [u32],
    #[spirv(push_constant)] constants: &PushConstants,
) {
    let idx = id.x as usize;
    if idx >= constants.num_particles as usize {
        return;
    }

    let mut p = particles[idx];

    let dir = p.rng.next() >> 30;

    let mut x = p.pos_x as i32;
    let mut y = p.pos_y as i32;

    match dir {
        0 => y -= 1,
        1 => x += 1,
        2 => y += 1,
        _ => x -= 1,
    }

    let max_x = (constants.grid_width - 1) as i32;
    let max_y = (constants.grid_height - 1) as i32;

    if x < 0 {
        x = 1;
    }
```

```
    if x > max_x {
        x = max_x - 1;
    }
    if y < 0 {
        y = 1;
    }
    if y > max_y {
        y = max_y - 1;
    }

    p.pos_x = x as u32;
    p.pos_y = y as u32;
    particles[idx] = p;

    let grid_idx = (p.pos_y * constants.grid_width + p.pos_x) as usize;
    unsafe {
        spirv_std::arch::atomic_i_add::<u32, 1, 0>(&mut grid[grid_idx],
1);
    }
}

#[spirv(vertex)]
pub fn fullscreen_vs(#[spirv(vertex_index)] vert_id: i32,
#[spirv(position)] out_pos: &mut Vec4, #[spirv(location = 0)] out_uv:
&mut Vec2) {
    const POSITIONS: [Vec4; 6] = [
        Vec4::new(-1.0, 1.0, 0.0, 1.0),
        Vec4::new(1.0, -1.0, 1.0, 0.0),
        Vec4::new(1.0, 1.0, 1.0, 1.0),
        Vec4::new(-1.0, 1.0, 0.0, 1.0),
        Vec4::new(-1.0, -1.0, 0.0, 0.0),
        Vec4::new(1.0, -1.0, 1.0, 0.0),
    ];
    let pos = POSITIONS[vert_id as usize];
    *out_pos = Vec4::new(pos.x, pos.y, 0.0, 1.0);
    *out_uv = Vec2::new(pos.z, pos.w);
}

#[spirv(fragment)]
pub fn grid_fs(
    #[spirv(location = 0)] in_uv: Vec2,
    #[spirv(storage_buffer, descriptor_set = 0, binding = 1)] grid:
&[u32],
    #[spirv(push_constant)] constants: &PushConstants,
    #[spirv(location = 0)] out_color: &mut Vec4,
) {
    let x = ((in_uv.x * constants.grid_width as f32) as
u32).min(constants.grid_width - 1);
```

```
    let y = ((in_uv.y * constants.grid_height as f32) as
u32).min(constants.grid_height - 1);

    let grid_idx = (y * constants.grid_width + x) as usize;
    let density = grid[grid_idx];
    let intensity = (density as f32 / constants.max_density as
f32).clamp(0.0, 1.0);

    *out_color = Vec4::new(intensity, 0.0, 0.0, 1.0);
}
```